Solving Hidato Puzzles Using Multiple AI Search Strategies:

A Comparative Analysis

Shantesh Vinayka

Aravind Shyam Kattepur

CS 5100: Foundations of Artificial Intelligence

Northeastern University

Professor Rose Sloan

Abstract

This paper presents a comparative analysis of five artificial intelligence search algorithms applied to solving Hidato, a number-placement puzzle defined on a hexagonal grid where consecutive integers must be placed in adjacent cells. The approaches evaluated include Depth-First Search (DFS), DFS with heuristic guidance, Constraint Satisfaction with propagation (CSP), A* Search, and a Genetic Algorithm (GA). These methods are tested on a benchmark suite covering different board sizes, clue densities, and clue placement patterns. The results indicate that CSP is the most efficient approach, solving most puzzles while exploring significantly fewer nodes than DFS. DFS serves as a strong baseline capable of handling most cases within the time limit. In contrast, A* and GA demonstrate significant limitations on larger and more complex puzzles. These findings are consistent with prior

work by Crawford et al. (2007), which highlights the importance of search strategy design in combinatorial problem solving.

1 Introduction

Hidato is a logic puzzle created by the Israeli mathematician Dr. Gyora Benedek. It is played on a hexagonal grid where the objective is to fill every cell with consecutive integers from 1 to N, such that each number is placed in a cell adjacent to the next number in the sequence. Depending on their position, cells may have up to six neighbors. Some cells are pre-filled with clue numbers that anchor portions of the solution, and the remaining values must be inferred.

From a computational perspective, Hidato is challenging due to the rapid growth of the search space as grid size increases. Unlike Sudoku, which enforces uniqueness constraints across rows, columns, and subgrids, Hidato requires constructing a valid path through the grid. This makes it structurally similar to a Hamiltonian path problem with additional constraints. As a result, strategies effective for Sudoku do not necessarily transfer directly to Hidato.

Crawford et al. (2007) investigated enumeration strategies in constraint programming using puzzles such as Sudoku, Magic Squares, and Latin Squares. Their work demonstrated that heuristic strategies, particularly those based on Minimum Domain Size (MiD), improve performance by prioritizing highly constrained variables. These findings motivate the exploration of different search approaches for Hidato, where constraints are sequential and adjacency-based.

The objective of this study is to systematically evaluate multiple AI search strategies for solving Hidato puzzles and to analyze how puzzle characteristics influence algorithm performance. The primary contribution of this work is a comparative evaluation across diverse puzzle configurations, including adversarial instances designed to highlight algorithmic

limitations.

2 Methods

2.1 Problem Formulation

A Hidato puzzle is defined on a hexagonal grid containing N cells. Each cell must be assigned a unique integer from 1 to N, with the constraint that consecutive integers occupy adjacent cells. Some cells contain fixed values that cannot be altered, while the remaining cells must be determined by the solver. The board is represented as a two-dimensional array, where each cell is either fixed, assigned, or empty. Adjacency relationships are computed based on hex-grid neighbor definitions.

2.2 System Architecture

The system is implemented as a full-stack application. The frontend is developed using React with TypeScript and provides an interactive interface for visualizing puzzle states and solver execution. The backend is implemented in Python and exposes a REST API. Each solver is implemented as a modular component that integrates with shared backend utilities, allowing for extensibility without modifying existing code.

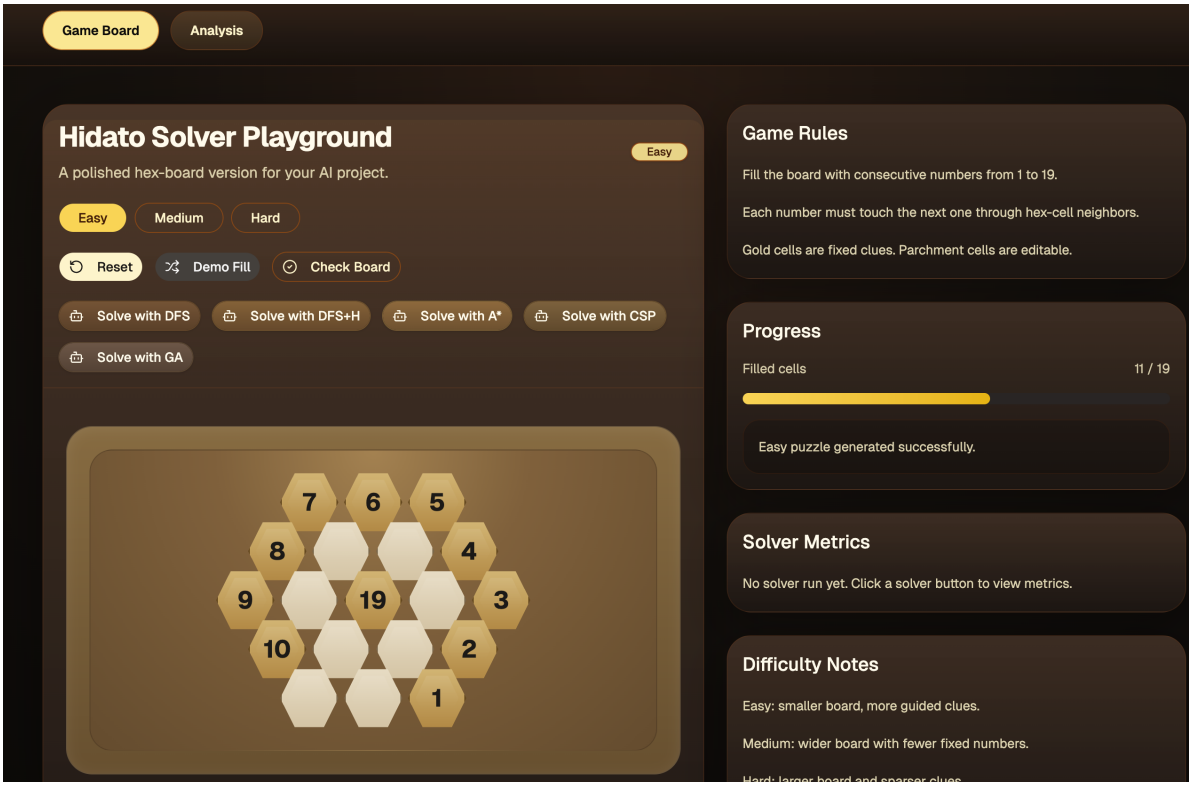


Figure 1: Interactive Hidato solver interface showing puzzle generation, difficulty selection, and solver execution options.

As shown in Figure 1, the frontend provides an interactive puzzle board together with controls for selecting difficulty levels and executing multiple AI solvers. This interface supports direct experimentation with the implemented methods and offers a clear visualization of puzzle state and solver interaction.

2.3 Algorithm Implementations

Depth-First Search (DFS): This baseline approach performs recursive backtracking by placing values sequentially. It explores valid placements and backtracks when constraints are violated. While it guarantees completeness, it may require exploring a large number of nodes.

DFS with Heuristic: This variant of DFS prioritizes cells with fewer valid future placements, inspired by the Minimum Remaining Values principle. The goal is to reduce

unnecessary exploration by focusing on constrained regions earlier in the search.

Constraint Satisfaction with Propagation (CSP): In this formulation, each cell is treated as a variable with a domain of possible values. Constraint propagation is applied to eliminate inconsistent assignments before search branching. This significantly reduces the search space by pruning invalid possibilities early.

A* Search: A* uses an evaluation function $f(n) = g(n) + h(n)$, where $g(n)$ represents progress and $h(n)$ estimates remaining effort. The heuristic considers adjacency constraints; however, the overhead of maintaining priority queues and computing heuristics impacts performance.

Genetic Algorithm (GA): The GA maintains a population of candidate solutions and evolves them through selection, crossover, and mutation. Each candidate is evaluated based on how well it satisfies adjacency constraints. However, due to the strict nature of these constraints, convergence is difficult.

2.4 Benchmark Suite and Adversarial Analysis

The benchmark suite varies three factors: board size (small, medium, large), clue density (approximately 20% and 30% of cells pre-filled), and clue placement pattern (random, evenly spread, or clustered). Lower clue ratios make puzzles more difficult because the solver receives less guidance. Clustered clue placement is especially difficult because it leaves large regions of the board with little local information. Each parameter combination is evaluated using multiple random seeds to avoid over-reliance on any single instance.

Adversarial analysis is also performed by generating puzzle configurations specifically designed to stress the solvers. This makes it possible to identify worst-case behavior for each algorithm and to observe where each method breaks down. Difficulty is labeled as easy, medium, or hard based on a composite of these parameters.

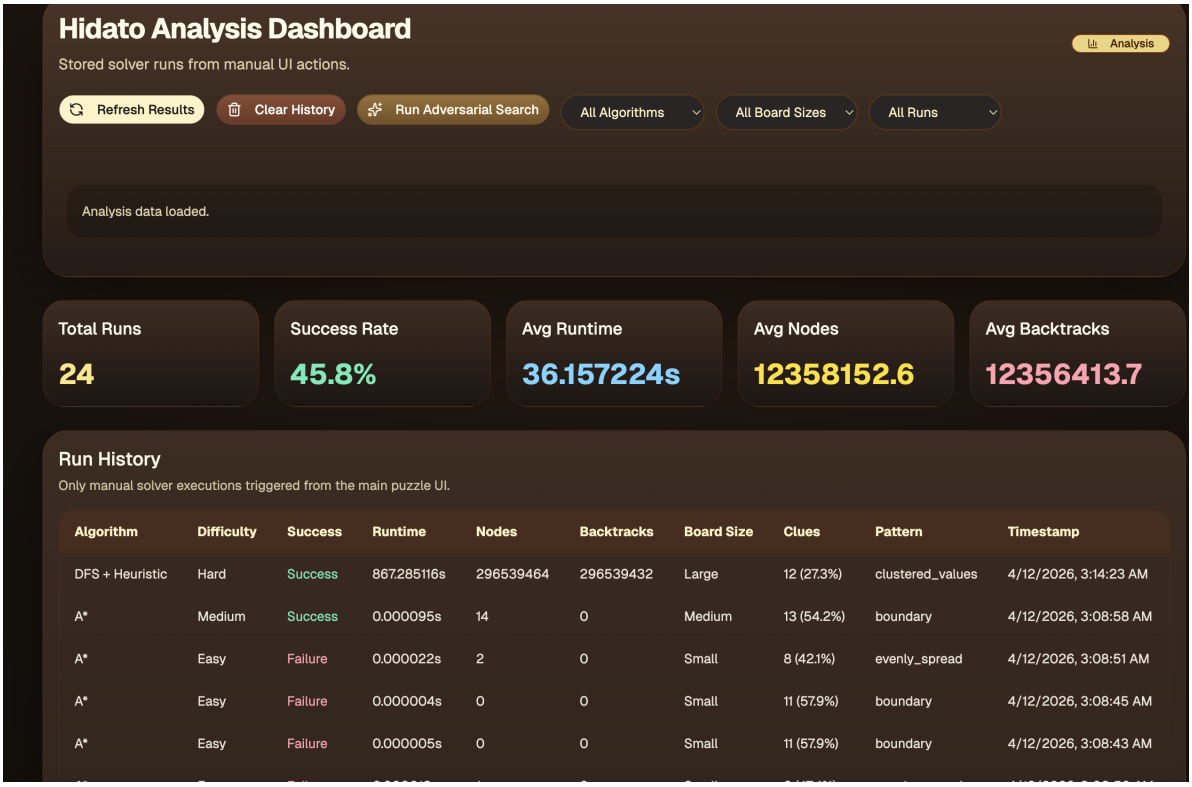
2.5 Evaluation Metrics

Performance is evaluated using runtime, number of nodes explored, number of backtracks, and success rate within a 15-second time limit. For A* and GA, which do not backtrack in the same way as DFS-based methods, the backtrack count is zero. A composite hardness score is also computed in order to rank puzzle difficulty across algorithms and configurations.

3 Results

3.1 Overall Solve Rates

The results indicate that both DFS and CSP achieve high solve rates across most benchmark configurations, including medium and large puzzles. DFS with heuristic guidance performs slightly worse, failing on certain difficult instances. A* fails on most hard puzzles, while the Genetic Algorithm does not successfully solve large-board instances.



Algorithm	Difficulty	Success	Runtime	Nodes	Backtracks	Board Size	Clues	Pattern	Timestamp
DFS + Heuristic	Hard	Success	867.285116s	296539464	296539432	Large	12 (27.3%)	clustered_values	4/12/2026, 3:14:23 AM
A*	Medium	Success	0.000095s	14	0	Medium	13 (54.2%)	boundary	4/12/2026, 3:08:58 AM
A*	Easy	Failure	0.000022s	2	0	Small	8 (42.1%)	evenly_spread	4/12/2026, 3:08:51 AM
A*	Easy	Failure	0.000004s	0	0	Small	11 (57.9%)	boundary	4/12/2026, 3:08:45 AM
A*	Easy	Failure	0.000005s	0	0	Small	11 (57.9%)	boundary	4/12/2026, 3:08:43 AM

Figure 2: Analysis dashboard displaying aggregate solver metrics and run history across multiple puzzle configurations.

Figure 2 shows the analysis dashboard used to review solver behavior across runs. The dashboard summarizes metrics such as success rate, runtime, nodes explored, and backtracks, and it also records individual run histories for comparison across puzzle settings.

3.2 Performance on Hard Puzzles

The most difficult puzzles are characterized by large grid sizes, low clue density, and clustered clue placement. In one particularly challenging instance, four out of five algorithms failed to find a solution. DFS successfully solved this instance, requiring approximately 2.2 million node explorations and 2.5 seconds of runtime, while CSP exceeded the time limit. Table 1 presents the top 10 hardest puzzles by average hardness score.

Table 1: Top 10 Hardest Puzzles by Average Hardness Score

Rank	Board	Clue Ratio	Pattern	Difficulty	Failures	Avg Score	Max Score
1	Large	0.20	Clustered	Hard	4	1162.88	1336.56
2	Large	0.20	Clustered	Hard	3	864.25	1336.56
3	Large	0.30	Clustered	Hard	2	771.12	1336.56
4	Large	0.20	Random	Hard	2	713.80	1275.98
5	Medium	0.29	Clustered	Hard	2	662.64	1327.67
6	Large	0.30	Random	Hard	2	636.22	1280.14
7	Medium	0.21	Random	Hard	2	589.80	1243.51
8	Large	0.30	Even Spread	Hard	2	565.60	1266.82
9	Large	0.30	Even Spread	Hard	2	531.48	1280.40
10	Large	0.30	Clustered	Hard	1	640.77	1278.67

Large puzzles with clustered clues dominate the ranking, with random distributions appearing next. Medium-sized boards appear only when they are paired with clustered clues, suggesting that clue placement pattern has a stronger effect than board size at the medium scale.

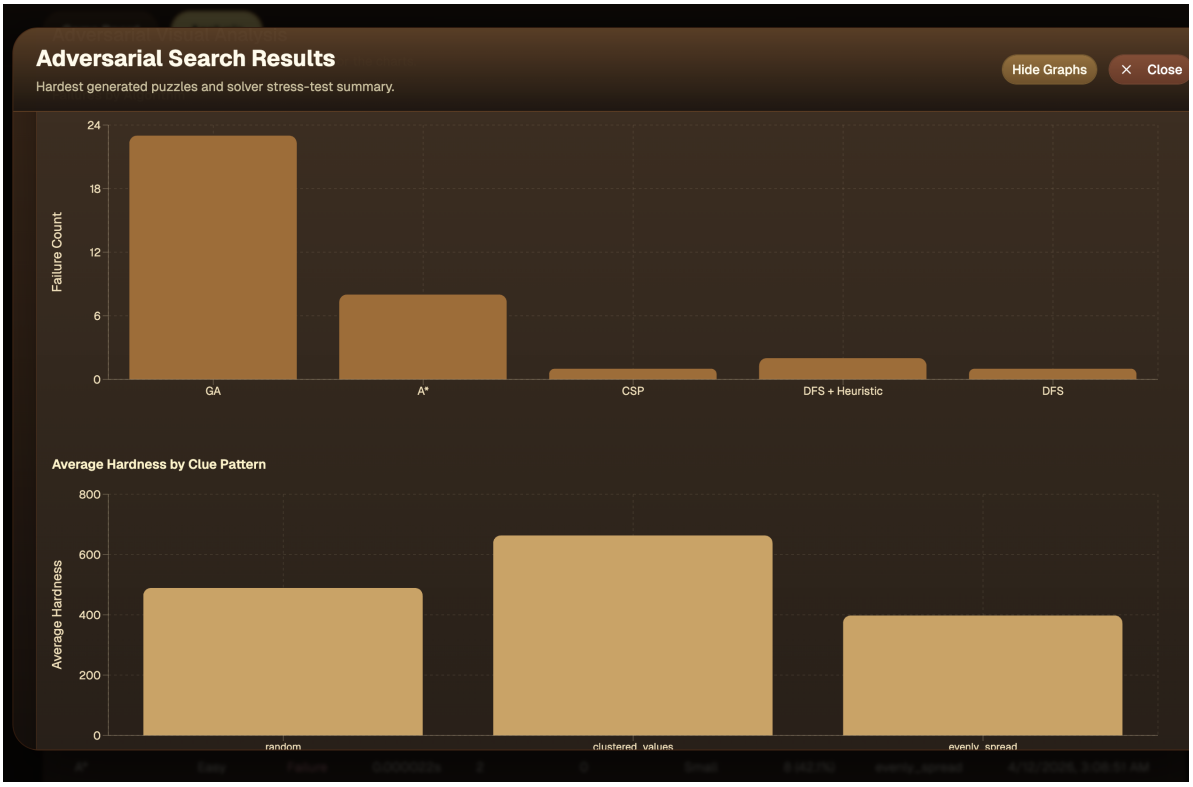


Figure 3: Adversarial analysis results showing solver failure counts and average hardness across clue placement patterns.

Figure 3 presents the results of adversarial analysis. The visual summary highlights how solver failures are distributed across algorithms and shows that clustered clue placements are associated with higher average hardness than random or evenly spread patterns.

3.3 Per-Algorithm Analysis

DFS: DFS performs reliably across most puzzles and solves all but one large clustered puzzle within the 15-second time limit. On hard puzzles that it solves, node counts range from approximately 11,000 to 3.5 million, while runtimes range from 0.01 to 4.5 seconds. Nodes and backtracks remain nearly identical because DFS backtracks on most dead ends. Its main weakness is performance variability depending on whether the correct path is explored early or late.

CSP: CSP is the most efficient solver by a wide margin. On puzzles where both DFS and

CSP succeed, CSP explores substantially fewer nodes. For example, on one large random-clue puzzle, DFS requires 136,776 nodes, while CSP solves the same puzzle with only 2,812. On large evenly spread puzzles, CSP sometimes solves instances with fewer than 200 nodes where DFS requires tens of thousands. Although it times out on one particularly difficult large clustered puzzle, it performs best overall.

DFS with Heuristic: This method improves performance on some puzzles but degrades it on others. On one large random puzzle, plain DFS requires 4.5 seconds and 3.5 million nodes, whereas heuristic DFS takes 7.1 seconds and 2.4 million nodes. Although the node count is lower, the runtime is longer because heuristic computation introduces overhead at each step. It also times out on two large clustered puzzles, compared with one for plain DFS. This suggests that the MRV-inspired approach does not transfer cleanly to Hidato's path-based structure.

A*: A* performs the worst among the systematic search methods. Eight of its ten hardest puzzles are outright failures. In many cases, failure occurs after exploring almost no nodes at all, sometimes only 0 or 13, indicating that the search cannot meaningfully begin. In several cases it terminates early because the heuristic fails to provide useful guidance. The two hardest puzzles it does solve require 12,704 and 626,996 nodes, respectively.

GA: The Genetic Algorithm fails on every large-board puzzle. It evaluates between 16,000 and 35,000 candidate solutions per run but never converges to a valid solution. The difficulty is structural: Hidato's constraints are so strict that crossover and mutation frequently produce invalid offspring. A single misplaced number can invalidate the entire chain, and combining partial solutions is not effective. These results indicate that population-based methods are not well suited to this problem.

Table 2 summarizes algorithm performance on hard large-board puzzles.

Table 2: Algorithm Comparison on Hard Large-Board Puzzles

Algorithm	Hard Solve Rate	Avg Runtime (s)	Avg Nodes (solved)	Key Weakness
DFS	High	0.01–4.53	11K–3.5M	Variance
CSP	High	0.03–7.48	60–19K	Rare timeouts
DFS Heuristic	Moderate	0.06–7.68	28K–2.8M	Overhead
A*	Low	0.00–8.34	0–627K	Memory/heuristic
GA	None (large)	0.13–0.26	17K–35K	Cannot converge

3.4 Impact of Puzzle Parameters

Board size has the strongest effect on difficulty, with large boards being substantially harder than medium or small boards for every algorithm. Clue density is the next most important factor, as 20% clue puzzles are considerably more difficult than 30% clue puzzles. Among clue placement patterns, clustered values are consistently the most difficult because they leave large sections of the board without guidance. Evenly spread clues produce the easiest instances, while random placement falls between the two.

4 Conclusions

The results demonstrate that constraint propagation is highly effective for solving Hidato puzzles, as evidenced by the performance of the CSP approach. It solves most puzzles while exploring far fewer states than the other methods, largely because domain pruning eliminates dead ends before the solver invests substantial effort in them. These findings are consistent with Crawford et al. (2007), who showed that propagation combined with strong enumeration strategies performs well on other combinatorial puzzles.

DFS remains a reliable baseline due to its simplicity and low per-node computational cost. However, heuristic enhancements and evolutionary methods do not perform as effectively in this domain because the sequential adjacency constraints are highly restrictive. The heuristic

DFS approach does not consistently offset its additional computational overhead, while A* lacks a sufficiently informative heuristic. Similarly, the Genetic Algorithm is not well suited to a problem in which small errors can invalidate an entire solution.

Future work may explore improved heuristic design for A*, stronger propagation techniques such as Maintaining Arc Consistency, and hybrid approaches combining constraint reasoning with informed search strategies. Expanding the evaluation to larger grids and irregular board shapes would also provide a broader assessment of solver behavior.

Works Cited

[1] B. Crawford, M. Aranda, C. Castro, and E. Monfroy, "Enumeration strategies in constraint programming for solving puzzles," in *Fourth Congress of Electronics, Robotics and Automotive Mechanics (CERMA 2007)*, 2007, pp. 718–723.

[2] G. Benedek, "Hidato puzzle," 2007. [Online]. Available: `https://www.hidato.com`

[3] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

[4] Python Software Foundation, "Python Language Reference, version 3.12," 2024. [Online]. Available: `https://www.python.org`

[5] Meta Platforms, "React: A JavaScript library for building user interfaces," 2024. [Online]. Available: `https://react.dev`

[6] Evan You, "Vite: Next Generation Frontend Tooling," 2024. [Online]. Available: `https://vitejs.dev`

[7] GitHub repository for this project: `https://github.com/AravindSK31/hidato-ai-project`